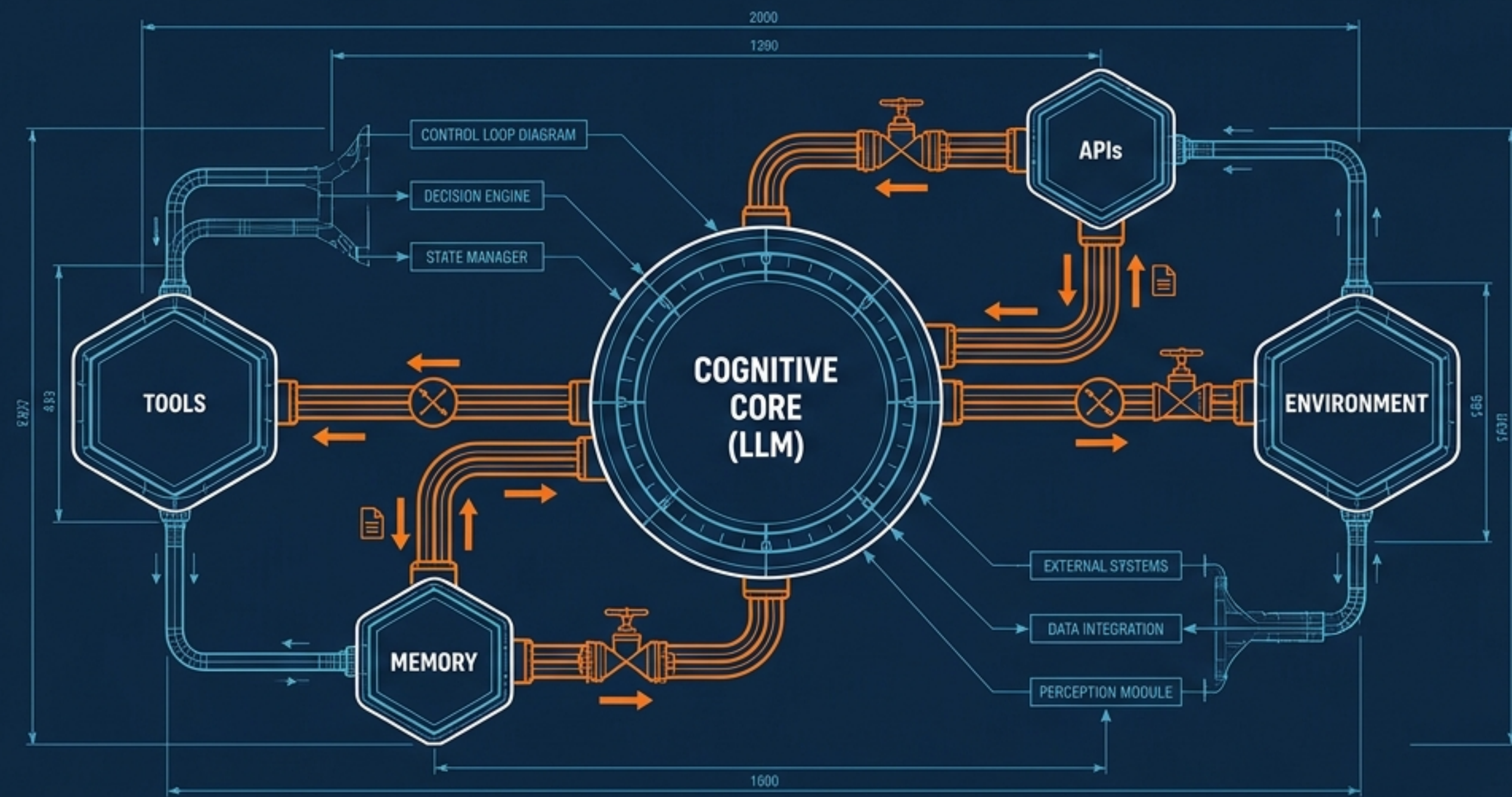


ENGINEERING AUTONOMOUS LLM SYSTEMS

A PRODUCTION-READY GUIDE FOR FUTURE DATA SCIENTISTS



PANEL SPEC: TECHNICAL OVERVIEW

SYSTEM INITIATION:

- Welcome to the foundations of agentic artificial intelligence.
- Transition from building passive chatbots to active problem-solvers.
- Master the architecture of autonomous, self-correcting workflows.
- Bridge the gap between raw language models and real-world tools.

PANEL SPEC: TECHNICAL OVERVIEW

SECTION:
INTRODUCTION

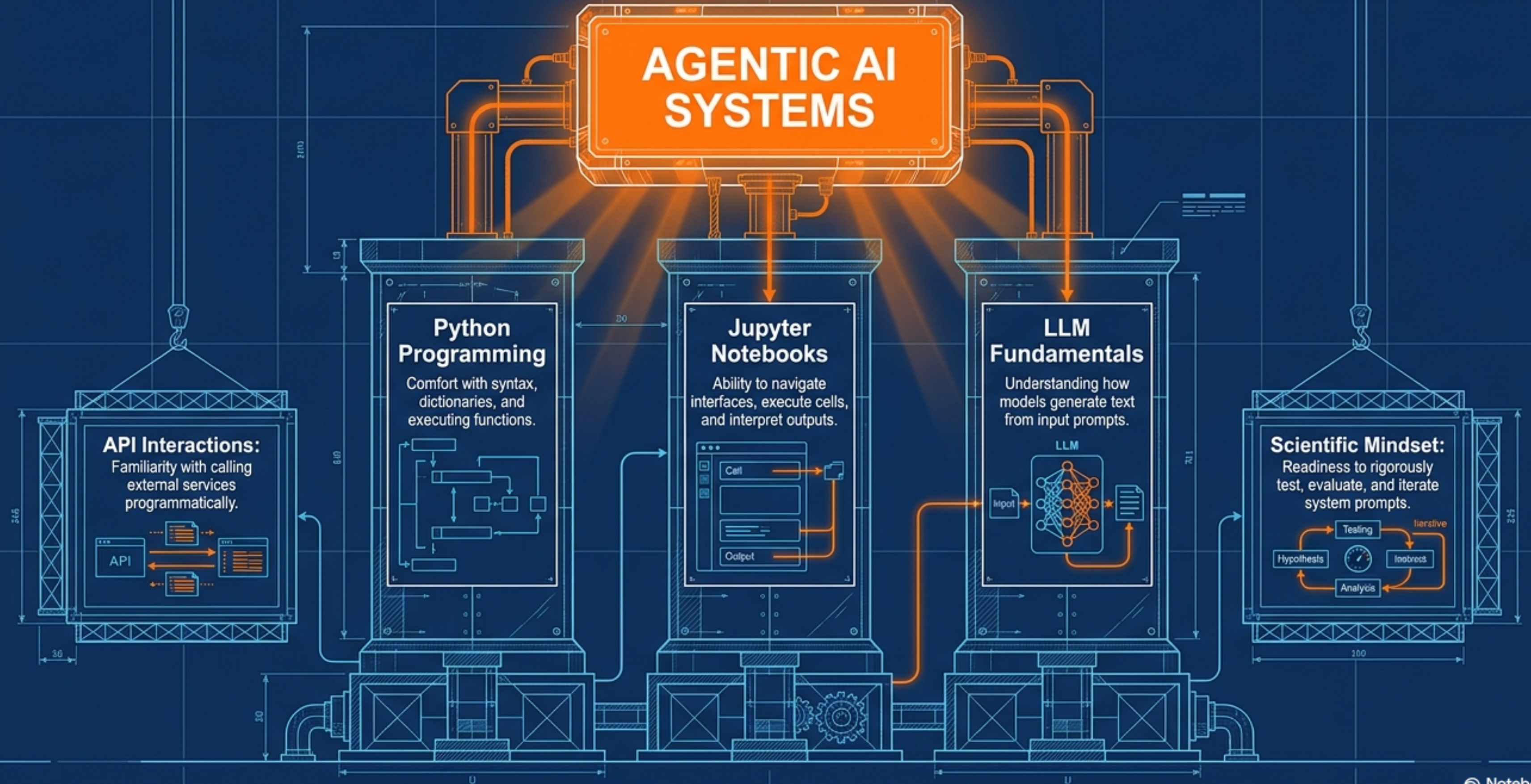
DRAWING NO:
EALS-001

SCHEMATIC DIAGRAM: A-1

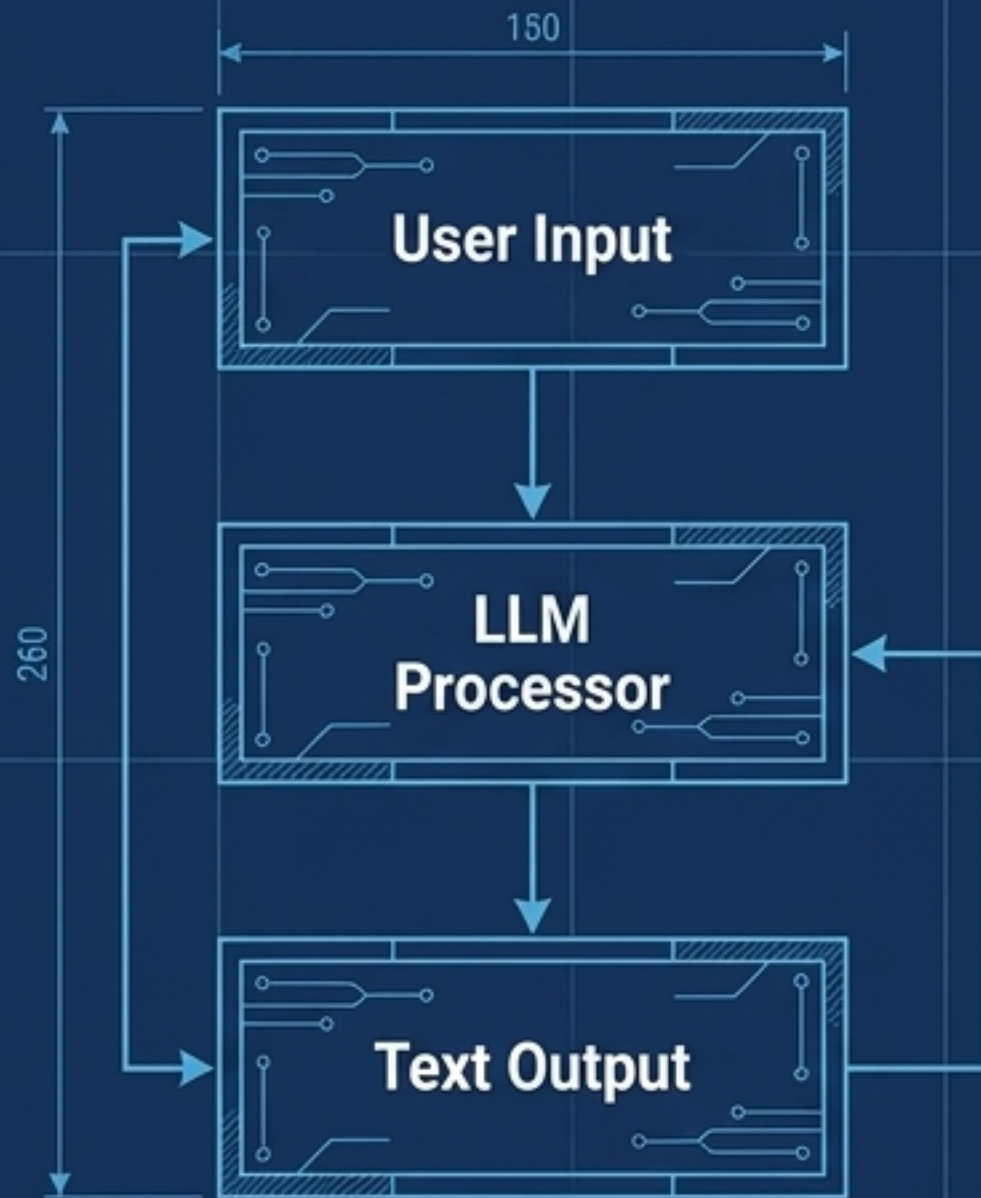
SCALE:
NTS

REVISION:
REVISION: 01 - AUTONOMOUS AGENT ARCHITECTURE

FOUNDATIONS FOR BUILDING AGENTIC AI



DEFINING THE AUTONOMOUS PROBLEM SOLVER



Passive text generators.
Reactive and strictly sequential.



Intelligent systems capable of
perceiving their digital environment.

Autonomy: They operate independently
toward a predefined goal.

Adaptability: They respond intelligently
to unexpected information or errors.

Interaction: They interface with databases,
APIs, and real-world tools.

BRIDGING THE CAPABILITY GAP

STANDALONE LLM

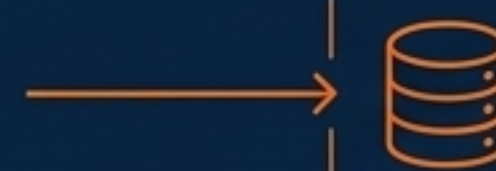
AI AGENT

KNOWLEDGE

Rely on static training data cut-offs.



Query live, real-time databases.



ACTION

Generate passive text responses.



Execute code and trigger external workflows.

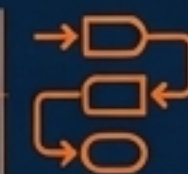


WORKFLOW

Handle single, isolated prompt interactions.

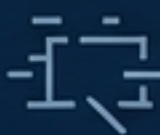


Dynamically plan and execute multi-step tasks.

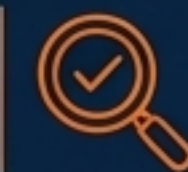


VERIFICATION

Prone to hallucination and logical leaps.



Observe tool results to verify factual accuracy.

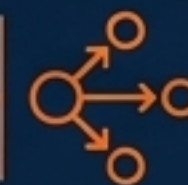


ROLE

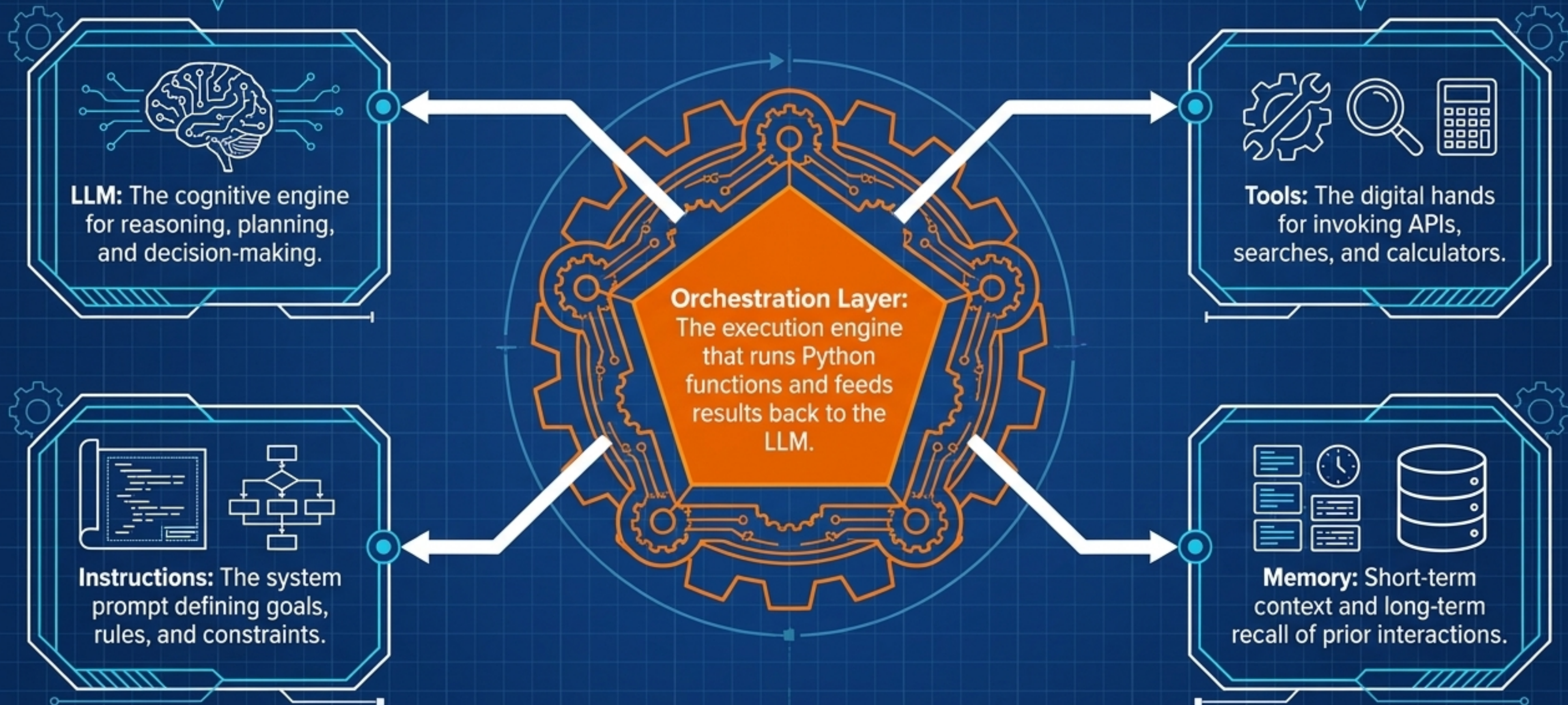
Reactive, isolated scholars.



Proactive, outward-facing cognitive collaborators.



ANATOMY OF AN AGENTIC SYSTEM



STRUCTURING AGENT INSTRUCTIONS

Prompting is the primary engineering lever for controlling agentic systems.

User Requests: Provide the specific task for the current interaction cycle.

Tool Descriptions:
Detail exactly what each tool does and its required parameters.
Clear, unambiguous descriptions are vital for correct API utilisation.

System Prompts:
Define the overall persona, constraints, and baseline behaviour.

DYNAMIC EXECUTION IN PRACTICE



Unlike a rigid script, the agent breaks the request into logical sub-tasks.



Identify intent: Customer requests refund for damaged product.

Fetch data: Execute tool to retrieve Order #12345 details.

Logic check: Compare retrieved product data against the 30-day return policy.

Execute: Reason that conditions are met and trigger the refund API tool.

CONSTRUCTING A ROBUST PERSONA

Prompting is the primary engineering lever for controlling agentic systems.

Define Persona

[ROLE]:

Act as a Senior Python Developer specialising in efficient algorithms.

[TASK]:

Write a function to parse the provided server logs.

[OUTPUT FORMAT]:

Return the parsed data strictly as a JSON object.

[EXAMPLES]:

Input: 'Error 404 at 10:00' -> Output: {'code': 404, 'time': '10:00'}

[CONTEXT]:

The logs are from an Nginx web server running in the London data centre.

Specify Goal

Dictate Structure

Provide Guidance

Establish Environment

Roles act as strong biases toward consistent, targeted, and predictable outputs, supplying background data not found in the model's training sets.

AUTOMATING PERSONA EVALUATION



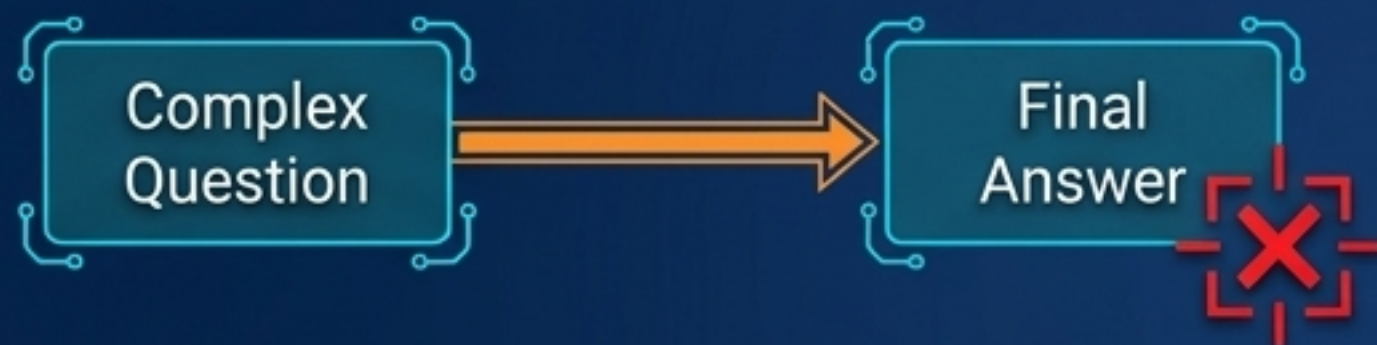
Rigorous Testing: Defining a persona requires rigorous rehearsal and ground truth comparison.

Beyond String Matching: Exact string matching fails when evaluating freeform text generation.

Objectivity: Provide the evaluator with a structured scoring rubric to ensure impartial assessment.

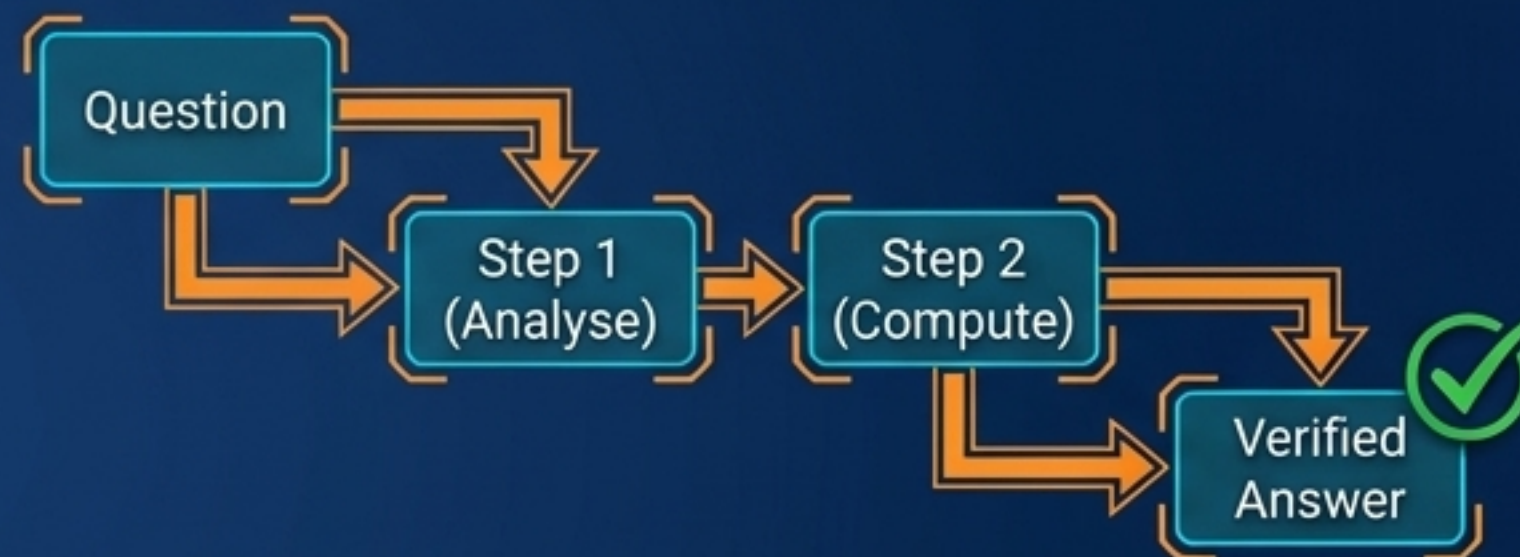
MAKING INTERNAL REASONING TRANSPARENT

DIRECT ANSWER (BLACK BOX)



Direct answers to complex problems often trigger logical errors or hallucinations.

CHAIN-OF-THOUGHT (CoT)



Forces the LLM to generate explicit intermediate steps, "thinking out loud" before answering.

ZERO-SHOT CoT

Simply appending "Think step by step" dramatically boosts accuracy.

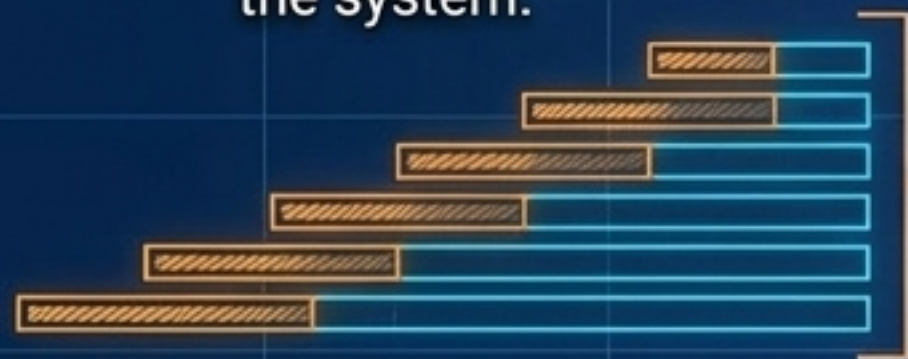
FEW-SHOT CoT

Providing sample reasoning chains teaches the LLM specific logical patterns.

THE ReAct LOOP: REASON AND ACT

WORKING MEMORY

Expanding horizontal bars representing "Working Memory", visually demonstrating that each cycle adds new context data to the system.



THOUGHT

LLM internally reasons about the next logical step required.

OBSERVATION

Orchestrator feeds the real-world tool results back to the LLM.

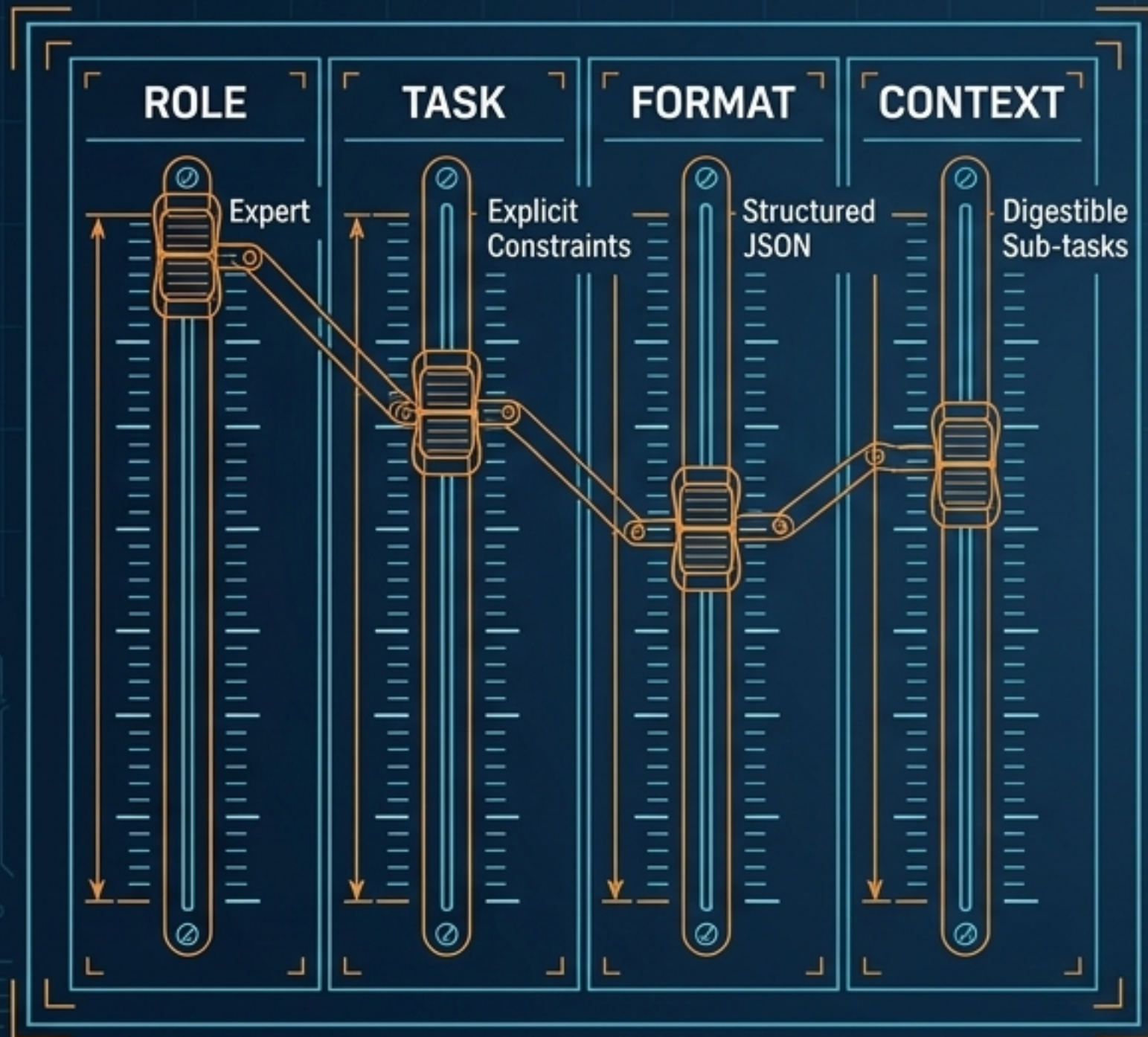
ACTION

LLM outputs a structured tool call for the orchestrator to execute.

TERMINATION

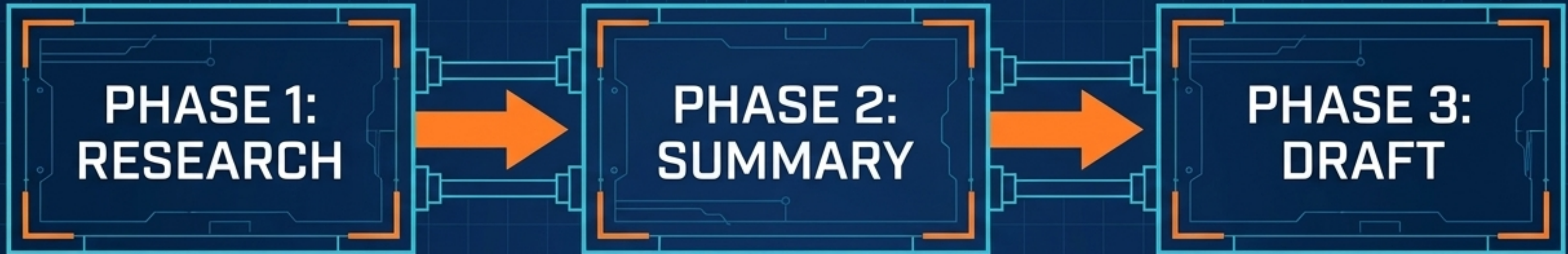
The loop concludes using a specific 'final_answer' tool, granting full context of the execution.

THE SCIENCE OF PROMPT REFINEMENT



- Engineering is an iterative, scientific process of observation and adjustment.
- Modify only one component at a time to isolate variables.
- Provide explicit constraints rather than assuming the model will infer intent.
- Break highly complex prompts into smaller, digestible sub-tasks.
- Avoid overwhelming the context window with contradictory rules.

STRUCTURING AGENTIC WORKFLOWS



The Domino Effect: Single prompts fail when juggling massive tasks like researching, drafting, and coding.

Sequential Prompting: Divides overarching goals into dedicated, modular steps.

Explicit Handoffs: The direct output of step one becomes the explicit input for step two.

Modular Debugging: Allows developers to isolate, test, and debug individual links, mirroring human workflows.

SAFETY GATES AND FEEDBACK LOOPS



THE BLUEPRINT TO PRODUCTION AGENTS

- **Agents** combine LLM reasoning with real-world tools, memory, and orchestration.
- **Role-Based Prompts** ensure consistent, highly targeted, and reliable outputs.
- **CoT and ReAct** unlock transparent, multi-step problem solving and planning.
- **Prompt Chaining and Gate Checks** build robust, modular production workflows.
- **Feedback Loops** enable autonomous, iterative self-correction without human intervention.

